

Mise en place d'un pipeline CI/CD

Jenkins & SonarQube

Audit et sécurisation d'un microservice d'authentification Java

Khalid Filali-Mdarhri · Cyprien Broche · Yanis Benadrouche

Dépôt : github.com/Kfilalimdarhri/Projet-CI-

1. Contexte & objectif

L'équipe de développement a livré, dans l'urgence, un microservice d'authentification en Java (Maven). Le code fonctionne mais contient des failles de sécurité et de mauvaises pratiques. Notre mission, en tant qu'ingénieurs DevOps, comporte quatre volets :

- Construire un pipeline d'intégration continue avec **Jenkins** ;
- Interconnecter **SonarQube** pour auditer le code automatiquement à chaque *push* ;
- Corriger les failles identifiées pour passer le **Quality Gate** ;
- (Bonus) Conteneuriser l'application avec **Docker**.

Environnement : toute l'infrastructure tourne en conteneurs Docker (Jenkins, SonarQube, PostgreSQL) via un `docker compose up`, sans aucune installation sur les postes. Deux branches Git sont utilisées : `avant-corrections` (code d'origine, fautif) et `main` (code corrigé).

2. Le pipeline Jenkins

Le pipeline est décrit dans un `Jenkinsfile` versionné avec le code. Il s'exécute automatiquement à chaque *push* et enchaîne six étapes, dont deux peuvent stopper le build (compilation et Quality Gate) :

Étape	Commande	Rôle
1. Checkout	<code>checkout scm</code>	Récupère le code depuis GitHub.
2. Build	<code>mvn clean compile</code>	Compile le code Java. Échec ⇒ build rouge.
3. Test & Coverage	<code>mvn test</code>	Lance les tests unitaires ; JaCoCo mesure la couverture.
4. Analyse SonarQube	<code>mvn sonar:sonar</code>	Envoie le code + la couverture à SonarQube pour audit.
5. Quality Gate	<code>waitForQualityGate</code>	Attend le verdict de Sonar. Statut « Failed » ⇒ build rouge (arrêt).
6. Package & Docker	<code>mvn package</code> + <code>docker build</code>	Produit le JAR et l'image Docker (seulement si tout est vert).

3. État des lieux initial PIPELINE ROUGE

Premier passage du pipeline sur la branche `avant-corrections` : le build échoue au stade du **Quality Gate**. SonarQube remonte une **faille de sécurité** (le mot de passe en dur) ainsi que plusieurs *code smells*. Le Quality Gate est au statut « **Failed** », ce qui bloque toute la suite de la chaîne.

4. Journal des corrections

Chaque problème remonté par SonarQube, son impact, et la modification apportée au code Java :

Problème détecté	Type	Pourquoi c'est un risque	Correction appliquée
Mot de passe de base de données en dur (<code>DB_PASSWORD = "super_secret..."</code>)	Faille critique	Quiconque accède au dépôt (fuite GitHub, stagiaire) obtient le mot de passe de production. Impossible à changer sans recompiler.	Suppression du secret du code : il est désormais lu depuis une variable d'environnement (<code>System.getenv("DB_PASSWORD")</code>).
Injection SQL par concaténation (<code>"... WHERE username = '" + username + "'"</code>)	Faille	Un utilisateur malveillant peut injecter du SQL et lire/détruire la base.	Passage à une requête préparée (<code>PreparedStatement</code> avec <code>?</code>), les entrées ne sont plus interprétées comme du code.
<code>System.out.println(...)</code> pour journaliser	Code smell	Pas de niveaux de log, pas d'horodatage, pas de traçabilité en production.	Remplacé par un <code>java.util.logging.Logger</code> (niveaux info / warning / severe).
Code mort : variables inutilisées (<code>unusedCounter</code> , <code>isLoggedIn</code>), imports inutiles, division par zéro factice, méthode <code>complexMethod</code>	Code smell	Augmente la complexité, masque les vrais bugs, rend la maintenance plus coûteuse.	Suppression complète du code mort et de la méthode trop complexe.
Blocs <code>catch</code> vides + gestion manuelle des ressources (<code>finally</code>)	Code smell	Les erreurs sont avalées en silence ; risque de fuite de connexions.	Gestion via <code>try-with-resources</code> (fermeture automatique) et journalisation des exceptions.

Exemple clé : la faille critique (mot de passe en dur)

Avant — le secret est écrit en clair dans le code source :

```
// UserService.java (avant)
private String DB_PASSWORD = "super_secret_password_123!";
...
conn = DriverManager.getConnection(url, "root", DB_PASSWORD);
```

Après — le secret sort du code, lu depuis l'environnement :

```
// UserService.java (après)
String dbPassword = System.getenv("DB_PASSWORD");
...
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, dbPassword)) { ... }
```

Exemple : injection SQL

```
// avant – concaténation, injection possible
String query = "SELECT * FROM users WHERE username = '" + username + "'";
rs = stmt.executeQuery(query);
```

```
// après – requête paramétrée
String query = "SELECT * FROM users WHERE username = ?";
PreparedStatement stmt = conn.prepareStatement(query);
stmt.setString(1, username);
```

5. Résultat final PIPELINE VERT

Après corrections (branche `main`), le pipeline se déroule entièrement : compilation, tests, analyse SonarQube, Quality Gate, puis création du JAR et de l'image Docker. La faille de sécurité a disparu, les code smells sont résolus, et le **Quality Gate passe au vert (« Passed »)**. Le build Jenkins est en succès sur tous ses stages.

6. Bonus — Conteneurisation Docker

Le pipeline produit une image Docker `epsi/bad-practices-app:latest` prête à déployer, à partir du `Dockerfile` du projet. Le conteneur est exécutable localement (`docker run`) et vérifiable via `docker ps` , garantissant un déploiement reproductible d'un environnement à l'autre.

7. Bilan CI/CD

Au-delà de la correction de ce code précis, le pipeline mis en place est un **garde-fou permanent**. À chaque *push*, il recompile, exécute les tests, audite la qualité et la sécurité, et **refuse automatiquement** (build rouge) tout code qui réintroduirait une faille ou une mauvaise pratique. L'équipe de développement conserve sa vitesse, mais ne peut plus mettre la production en danger sans que la chaîne le détecte et le bloque. La conteneurisation garantit en prime un déploiement reproductible, identique d'un poste à l'autre.